Lecture 15: Diffusion models

Last time we developed the **variational autoencoder (VAE)**. The VAE follows the usual recipe. We first specify the following two ingredients:

- **prior**: $p_\theta(\mathbf{z})$
- **likelihood**: $p_\theta(\mathbf{x} \mid \mathbf{z})$ (i.e., the **"decoder"**)

With these two pieces specified, the posterior is implied, though intractable. We then developed an amortized variational EM algorithm to approximate it:

- **posterior**: $q_\phi(\mathbf{z} \mid \mathbf{x}) \approx p_\theta(\mathbf{z} \mid \mathbf{x})$ (i.e., the **"encoder"**)

Again this follows the standard Bayesian recipe: *specify* prior and likelihood, then perform approximate posterior inference. However, what if we ultimately *only* care about learning a good generative model $\mathbf{x} \sim p_\theta(\mathbf{x})$? In other words, we do not care about the posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$, we just want the ability to generate realistic synthetic $\mathbf{x}$. Perhaps one thing we could do instead is to specify:

- **prior** $p_\theta(\mathbf{z})$,
- **"posterior"** $q_\phi(\mathbf{z} \mid \mathbf{x}) \approx p_\theta(\mathbf{z} \mid \mathbf{x})$

Then with these two pieces specified, a (family of) likelihood(s) is implied, since of these three pieces (prior, likelihood, posterior) there are only two degrees of freedom: $p_\theta(\mathbf{z} \mid \mathbf{x}) \propto p_\theta(\mathbf{z})\, p_\theta(\mathbf{x} \mid \mathbf{z})$

That is the idea behind **denoising diffusion models**. We will specify a noising process $q(\mathbf{z} \mid \mathbf{x})$ upfront, which maps a data point $\mathbf{x}$ to a "latent code" $\mathbf{z}$, and we will specify it in very *particular* way which then makes learning $p_\theta(\mathbf{x} \mid \mathbf{z})$ easy.

This lecture drawns on the very nice note by Luo (2022) "Understanding Diffusion Models: A Unified Perspective", which works through the math of three equivalent views on diffusion models:

1. diffusion models as **hierarchical VAEs**
2. diffusion models as **stochastic differential equations (SDEs)**
3. diffusion models as **score-based generative models**

There many different kinds of diffusion models nowadays. But this lecture will only focus on the Gaussian diffusion models introduced by Ho et al. (2020), Song et al. (2020), and Kingma et al. (2021).

The forward (noising) process $q(\mathbf{z} \mid \mathbf{x})$

Diffusion models begin by specifying upfront the form for $q(\mathbf{z} \mid \mathbf{x})$. This is called the **forward process** and it will be **a fixed Markov chain that iteratively adds Gaussian noise to $\mathbf{x}$**. Concretely: let $\mathbf{z}_0 \equiv \mathbf{x}$ be the data, and define a sequence of latents $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T$ via the transitions

$$q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) \;=\; \mathcal{N}(\sqrt{\alpha_t}\,\mathbf{z}_{t-1},\, (1-\alpha_t)\, I) \qquad t = 1, \ldots, T$$

where $\alpha_t \in (0, 1)$ is a fixed schedule of noise rates. Moreover we select $\alpha_{1:T}$ such that:

$$q(\mathbf{z}_T \mid \mathbf{x}) \approx \mathcal{N}(\mathbf{z}_T;\, \mathbf{0}, I)$$

In other words, we iteratively noise the input until no information is left and the marginal over $\mathbf{z}_T$ is white noise.

The joint forward "encoder" is thus:

$$q(\mathbf{z} \mid \mathbf{x}) \equiv q(\mathbf{z}_{1:T} \mid \mathbf{z}_0) \;=\; \prod_{t=1}^{T} q(\mathbf{z}_t \mid \mathbf{z}_{t-1})$$

- All latents $\mathbf{z}_t$ live in the same space as $\mathbf{x}$ (same dimension D). There is no "bottleneck."
- The encoder q has **no parameters to learn.** It is just a pre-specified noise schedule. That's the point!

The forward marginals $q(\mathbf{z}_t \mid \mathbf{z}_0)$

Recall the reparameterization of the Gaussian: if $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ then $\mathbf{x} = \boldsymbol{\mu} + \Sigma^{1/2}\boldsymbol{\epsilon}$ for $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$. Each step of the forward process can then be written as:

$$\mathbf{z}_t \;=\; \sqrt{\alpha_t}\,\mathbf{z}_{t-1} \;+\; \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, I)$$

Substituting recursively and using the fact that the sum of independent Gaussians is Gaussian, after t steps:

$$\mathbf{z}_t \;=\; \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0 \;+\; \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I), \qquad \bar{\alpha}_t \;\equiv\; \prod_{s=1}^{t} \alpha_s$$

Equivalently:

$$q(\mathbf{z}_t \mid \mathbf{z}_0) \;=\; \mathcal{N}\big(\mathbf{z}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0,\, (1-\bar{\alpha}_t)\,I\big)$$

So for any noise level t, we can sample $\mathbf{z}_t$ given $\mathbf{z}_0$ in one shot, without simulating the full chain.

The reverse (denoising) process $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t+1})$

Having specified the forward (noising) process $q(\mathbf{z}_{1:T} \mid \mathbf{z}_0)$ we can now think of the *reverse* direction. At each step, can we take a noisy sample $\mathbf{z}_t$ and produce $\mathbf{z}_{t-1}$, a *slightly less noisy* version? If so, then iterating it gives a recipe for **generating** $\mathbf{x}$ from white noise $\mathbf{z}_T$.

Consider the following "complete conditional" of $\mathbf{z}_{t-1}$:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \;=\; \frac{q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x})\, q(\mathbf{z}_{t-1} \mid \mathbf{x})}{q(\mathbf{z}_t \mid \mathbf{x})}$$

Every term on the right-hand side is a known Gaussian:

$$q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}) = q(\mathbf{z}_t;\, \mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t \mid \sqrt{\alpha_t}\,\mathbf{z}_{t-1},\, (1-\alpha_t)\,I) \tag{1}$$

$$q(\mathbf{z}_{t-1} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_{t-1};\, \sqrt{\bar{\alpha}_{t-1}}\,\mathbf{x},\, (1-\bar{\alpha}_{t-1})\,I) \tag{2}$$

$$q(\mathbf{z}_t \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x},\, (1-\bar{\alpha}_t)\,I) \tag{3}$$

We could plug all these forms in to derive $q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x})$ (as Luo does eqs. 71–84) or we could simply appeal to Gaussian-Gaussian conjugacy. Either way, we will find that the conditional is Gaussian:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \;=\; \mathcal{N}\big(\mathbf{z}_{t-1};\, \boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}),\, \sigma_q^2(t)\,I\big)$$

where the mean and variance are the following functions:

$$\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) = \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})\,\mathbf{z}_t \;+\; \sqrt{\bar{\alpha}_{t-1}}\,(1-\alpha_t)\,\mathbf{x}}{1-\bar{\alpha}_t} \tag{4}$$

$$\sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \tag{5}$$

Great! We're done! But wait, the "reverse" conditional we just derived conditions on $\mathbf{x}$ which is the thing we are trying to generate. **What we *really* want is:**

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t) \;=\; \frac{q(\mathbf{z}_t \mid \mathbf{z}_{t-1})\, q(\mathbf{z}_{t-1})}{q(\mathbf{z}_t)}$$

In this case, only *one* of these terms is a known Gaussian. The other two are equal to:

$$q(\mathbf{z}_T) = \int q(\mathbf{z}_T \mid \mathbf{x}) q_{\text{data}}(\mathbf{x}) d\mathbf{x} \tag{6}$$

$$= \int \mathcal{N}(\mathbf{z}_T; \cdots) q_{\text{data}}(\mathbf{x}) d\mathbf{x} \tag{7}$$

where $q_{\text{data}}(\mathbf{x})$ is the (unknown) data distribution, exactly the thing we want fit and sample from. So we haven't really gotten anywhere.

However, the form of the conditional above will help us in specifying the form of a parametric "reverse" model p_θ whose parameters we will learn so that:

$$p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t) \;\approx\; q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$$

where the LHS side does not depend on $\mathbf{x}$. The reverse model will effectively learn to infer $\mathbf{x}$ from $\mathbf{z}_t$ and then output the appropriate reverse transition to $\mathbf{z}_{t-1}$.

Specifying $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t+1})$

What form should $p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)$ take? We can begin by specifying the top-level "prior" to be white noise:

$$p_\theta(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T; 0, I)$$

Since ultimately we will want to simulate from white noise, and then reverse the noise to produce $\mathbf{x}$.

Then for $t < T$, we will eventually (next section) seek to learn the parameters of a reverse process that minimize:

$$\min_\theta \mathrm{KL}(q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \,\|\, p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t))$$

It turns out, with this objective in mind, it will be convenient to specify p_θ to be:

$$p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t) \;=\; \mathcal{N}\big(\mathbf{z}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{z}_t, t),\, \sigma_q^2(t)\, I\big)$$

where $\boldsymbol{\mu}_\theta(\mathbf{z}_t, t)$ is a neural network with parameters θ shared across all timesteps t and $\sigma_q^2(t)$ is the (known) variance term that we derived above for the conditional under the forward process. Under this formulation, the only thing we will seek to learn is the mean function. By specifing the reverse model in this way, the KL divergence above becomes something quite simple and nice:

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{\mu}_q(\mathbf{z_t}, \mathbf{x}),\, \sigma_q^2(t)\, I) \,\big\|\, \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}_t, t),\, \sigma_q^2(t)\, I)\big) \;=\; \frac{1}{2\sigma_q^2(t)} \left\|\boldsymbol{\mu}_q(\mathbf{z_t}, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\right\|^2$$

Maximizing the ELBO

With the forward chain (variational posterior) $q(\mathbf{z}_{1:T} \mid \mathbf{x})$ fixed and the reverse chain (generative model) $p_\theta(\mathbf{x}, \mathbf{z}_{1:T})$ parameterized, we are in familiar territory. Under our formulation the marginal $p_\theta(\mathbf{x})$ is intractable, but we can lower bound it with the ELBO:

$$\log p_\theta(\mathbf{x}) \;\geq\; \mathbb{E}_{q(\mathbf{z}_{1:T}|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z}_{1:T})}{q(\mathbf{z}_{1:T} \mid \mathbf{x})}\right] \equiv \mathcal{L}(\theta)$$

Substituting the factorizations of the reverse chain (numerator) and forward chain (denominator) reveals a certain structure:

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\left[\log \frac{p_\theta(\mathbf{z}_1, \mathbf{x})}{q(\mathbf{z}_1 \mid \mathbf{x})}\right] + \sum_{t=2}^{T-1} \mathbb{E}_{q(\mathbf{z}_t|\mathbf{z}_{t-1})}\left[\log \frac{p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)}{q(\mathbf{z}_t \mid \mathbf{z}_{t-1})}\right] + \mathbb{E}_{q(\mathbf{z}_T|\mathbf{z}_{T-1})}\left[\log \frac{p_\theta(\mathbf{z}_{T-1}, \mathbf{z}_T)}{q(\mathbf{z}_T \mid \mathbf{z}_{T-1})}\right]$$

Namely, that the ELBO can be viewed as the ELBO for a **hierarchical VAE** where $\mathbf{z}_t$ is the "data" and $\mathbf{z}_{t-1}$ is the latent.

Another way we can rewrite the ELBO is:

$$\mathcal{L}(\theta) \;=\; \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z}_1)\big]}_{\text{reconstruction}} \;-\; \underbrace{\mathbb{E}_{q(\mathbf{z}_{T-1}|\mathbf{x})}\big[\mathrm{KL}\big(q(\mathbf{z}_T \mid \mathbf{z}_{T-1}) \,\|\, p(\mathbf{z}_T)\big)\big]}_{\text{prior matching}} \;-\; \sum_{t=1}^{T-1} \underbrace{\mathbb{E}_{q(\mathbf{z}_{t-1},\mathbf{z}_{t+1}|\mathbf{x})}\big[\mathrm{KL}\big(q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) \,\|\, p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t+1})\big)\big]}_{\text{consistency}}$$

- **Reconstruction:** the final decoder log-likelihood ($\mathbf{z}_1 \to \mathbf{x}$). Same as the VAE reconstruction term.
- **Prior matching:** the very last step of the forward process should match the prior $\mathcal{N}(\mathbf{0}, I)$. By construction this is should be close to zero.
- **Consistency:** at each intermediate level t, the forward transition $q(\mathbf{z}_t \mid \mathbf{z}_{t-1})$ should agree with the reverse transition $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t+1})$. The two views of $\mathbf{z}_t$, one from below and one from above, should be "consistent".

Note that the consistency term conditions on *both* $\mathbf{z}_{t-1}$ and $\mathbf{z}_{t+1}$. We will eventually want to derive a Monte Carlo estimator for gradients of the ELBO. Ideally we would factorize the ELBO into expectations over only single variables, which will reduce the variance of gradient estimates. We can derive another decomposition:

$$\mathcal{L}(\theta) \;=\; \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z}_1)\big]}_{\text{reconstruction}} \;-\; \underbrace{\mathrm{KL}\big(q(\mathbf{z}_T \mid \mathbf{x}) \,\|\, p(\mathbf{z}_T)\big)}_{\text{prior matching}} \;-\; \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\big[\mathrm{KL}\big(q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \,\|\, p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)\big)\big]}_{\text{denoising matching}}$$

The reconstruction and prior-matching terms have the same interpretation as before. The middle "consistency" terms have been **replaced with denoising-matching terms**: each one compares the learned reverse $p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t)$ to the known conditional under the forward process $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$. Moreover, the expectations now only involve $\mathbf{z}_t$, which will lead to lower Monte Carlo variance when we introduce gradient estimators.

Moreover, notice the following:

- The **prior-matching term** is zero (by construction).
- The **reconstruction term** is the standard VAE reconstruction at the final denoising step.
- The **denoising matching terms** have a nice closed-form structure which we derived earlier.

$$\mathcal{L}(\theta) = \underbrace{\mathbb{E}_{q(\mathbf{z}_1|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z}_1)\big]}_{\text{reconstruction}} \;-\; \sum_{t=2}^{T} \underbrace{\tfrac{1}{2\sigma_q^2(t)}\mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\big[\|\boldsymbol{\mu}_q(\mathbf{z_t}, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2\big]}_{\text{denoising matching}} \tag{8}$$

$$\propto_\theta \; -\sum_{t=1}^{T} \tfrac{1}{2\sigma_q^2(t)}\, \mathbb{E}_{q(\mathbf{z}_t|\mathbf{x})}\big[\|\boldsymbol{\mu}_q(\mathbf{z_t}, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2\big] \tag{9}$$

So the ELBO is proportional to the simplest possible loss one could write down for this problem: a mean-squared error between the target mean $\boldsymbol{\mu}_q$ and the model's predicted mean $\boldsymbol{\mu}_\theta$. Moreover, viewing the terms $\frac{1}{2\sigma_q^2(t)}$ as specifying a weighting over time $q(t)$, we can rewrite the expectation as:

$$\mathcal{L}(\theta) \;\propto_\theta\; -\mathbb{E}_{t\sim q(t)}\mathbb{E}_{\mathbf{x}\sim q_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{z}_t\sim q(\mathbf{z}_t|\mathbf{x})}\big[\|\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2\big]$$

.

Empirically, Ho et al. (2020) report that dropping the per-t weights entirely and find that uniform weights $q(t) = \frac{1}{T}$ improves sample quality. Notice that we have added in an expectation over $\mathbf{x} \sim q_{\text{data}}(\mathbf{x})$, which is just a way of saying that we will be averaging over samples from the empirical distribution.

Reparameterization gradient estimator

Sample $\mathbf{x} \sim q_{\text{data}}$, $t \sim \mathcal{U}\{1, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$, and form $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$. The single-sample reparameterized gradient estimator of the (negative) ELBO is:

$$\widehat{\nabla}_\theta \mathcal{L} = n\, w(t)\, \nabla_\theta \left\|\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\right\|^2$$

for some weighting $w(t)$ of the time steps and for n data points. This is an unbiased estimator. And note that the gradient passes through the deterministic reparameterization $\mathbf{z}_t(\mathbf{x}, \boldsymbol{\epsilon})$.

Training procedure

```
Diffusion training (mean-prediction parameterization):

  initialize θ
  until convergence:
      sample mini-batch {x^(1), ..., x^(B)} from dataset
      for each x:
          sample t ~ Uniform{1, ..., T}
          sample ε ~ N(0, I)
          z_t = sqrt(ᾱ_t) * x + sqrt(1 - ᾱ_t) * ε                          # reparameterize z_t
          μ_q = [sqrt(α_t)(1 - ᾱ_{t-1}) * z_t + sqrt(ᾱ_{t-1})(1 - α_t) * x] / (1 - ᾱ_t)
          loss_b = || μ_θ(z_t, t) - μ_q ||²                                  # match target mean
      g = ∇_θ (1/B) Σ_b loss_b                                               # backprop
      Adam step
```

Notice what's striking. The training loop looks like **supervised regression**: noise the data at a random level, ask the network to predict the target reverse-mean $\boldsymbol{\mu}_q$, train with MSE. There's no inference network to learn, no posterior collapse, no amortization gap. The cleverness of q pays for itself: training a generative model has been reduced to denoising regression.

Three equivalent things to predict

The target $\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x})$ is a *known, fixed, linear* function of $\mathbf{z}_t$ and $\mathbf{x}$. So we can re-parameterize $\boldsymbol{\mu}_\theta(\mathbf{z}_t, t)$ in terms of any quantity that determines $\mathbf{x}$ given $\mathbf{z}_t$. The MSE loss $\|\boldsymbol{\mu}_q - \boldsymbol{\mu}_\theta\|^2$ then becomes a (re-weighted) MSE loss on *that* quantity. Two popular re-parameterizations:

1. Predict $\mathbf{x}$ directly. Let $\hat{\mathbf{x}}_\theta(\mathbf{z}_t, t)$ be a neural network predicting the clean data:

$$\|\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2 \;=\; \frac{\bar{\alpha}_{t-1}\,(1-\alpha_t)^2}{(1-\bar{\alpha}_t)^2}\,\left\|\mathbf{x} - \hat{\mathbf{x}}_\theta(\mathbf{z}_t, t)\right\|^2$$

So predicting $\boldsymbol{\mu}$ is the same as predicting $\mathbf{x}$, up to a t-dependent constant.

2. Predict the noise $\boldsymbol{\epsilon}$. Use the forward reparameterization $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ to express $\mathbf{x}$ in terms of $\boldsymbol{\epsilon}$, i.e. $\mathbf{x} = (\mathbf{z}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon})/\sqrt{\bar{\alpha}_t}$, and substitute into $\boldsymbol{\mu}_q$. After simplification (using $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$):

$$\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) \;=\; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{z}_t \;-\; \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right)$$

Let $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{z}_t, t)$ be a neural network predicting the noise and define $\boldsymbol{\mu}_\theta$ by the same expression with $\hat{\boldsymbol{\epsilon}}_\theta$ in place of $\boldsymbol{\epsilon}$. Then the $\mathbf{z}_t$ terms cancel and:

$$\|\boldsymbol{\mu}_q(\mathbf{z}_t, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2 \;=\; \frac{(1-\alpha_t)^2}{\alpha_t\,(1-\bar{\alpha}_t)}\,\left\|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{z}_t, t)\right\|^2$$

So predicting $\boldsymbol{\mu}$ is the same as predicting $\boldsymbol{\epsilon}$, up to a (different) t-dependent constant.

3. Predict the score $\nabla_{\mathbf{z}_t} \log q(\mathbf{z}_t)$. This we derive later.

The continuous limit: SDEs (sketch)

What if we send $T \to \infty$? Let's derive the SDE directly from the discrete forward step. Reparameterize the noise schedule so $\alpha_t = 1 - \beta(t)\,\Delta t$, where $\Delta t = 1/T$ is the (small) time-step size and $\beta(t) > 0$ is a continuous-time rate function. As $T \to \infty$, $\Delta t \to 0$, and each discrete step becomes infinitesimal. Substituting into the discrete update

$$\mathbf{z}_t \;=\; \sqrt{\alpha_t}\,\mathbf{z}_{t-1} \;+\; \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t$$

and Taylor-expanding the two square roots to first order in Δt,

$$\sqrt{\alpha_t} \;=\; \sqrt{1-\beta(t)\,\Delta t} \;\approx\; 1 - \tfrac{1}{2}\beta(t)\,\Delta t, \qquad \sqrt{1-\alpha_t} \;=\; \sqrt{\beta(t)\,\Delta t}$$

The step becomes:

$$\mathbf{z}_t - \mathbf{z}_{t-1} \;=\; -\tfrac{1}{2}\,\beta(t)\,\mathbf{z}_{t-1}\,\Delta t \;+\; \sqrt{\beta(t)\,\Delta t}\,\boldsymbol{\epsilon}_t$$

Now let $\Delta t \to 0$. The LHS becomes the infinitesimal $d\mathbf{z}_t$. On the RHS, $\beta(t)\,\Delta t \to \beta(t)\,dt$. The noise term is Gaussian with covariance $\beta(t)\,\Delta t I$ which becomes $\sqrt{\beta(t)}\,d\mathbf{W}_t$, the increment of a **standard Brownian motion** $\mathbf{W}_t$, a stochastic process whose increment over a time interval Δt is a centered Gaussian with covariance $\Delta t \cdot I$:

$$\mathbf{W}_{t+\Delta t} - \mathbf{W}_t \;\sim\; \mathcal{N}(\mathbf{0},\; \Delta t \cdot I)$$

We arrive at:

$$d\mathbf{z}_t \;=\; \underbrace{-\tfrac{1}{2}\,\beta(t)\,\mathbf{z}_t\,dt}_{\text{drift (deterministic)}} \;+\; \underbrace{\sqrt{\beta(t)}\,d\mathbf{W}_t}_{\text{diffusion (stochastic)}}$$

So the discrete Gaussian chain becomes, in the limit, a **continuous-time SDE**: Brownian motion coupled with a deterministic drift. The function $\beta(t)$ is the continuous-time analogue of the discrete schedule (with $\beta(t) \approx (1-\alpha_t)/\Delta t$).

The drift $-\frac{1}{2}\beta(t)\,\mathbf{z}_t\,dt$ has **the opposite sign of $\mathbf{z}_t$**. Wherever $\mathbf{z}_t$ sits, the drift points back toward the origin, with strength proportional to $|\mathbf{z}_t|$ and rate $\beta(t)$. Without noise, the SDE collapses to the ODE $d\mathbf{z}_t = -\frac{1}{2}\beta(t)\mathbf{z}_t\,dt$, which has solution

$$\mathbf{z}_t \;=\; \mathbf{z}_0\,\exp\!\left(-\tfrac{1}{2}\int_0^t \beta(s)\,ds\right)$$

which is pure exponential decay to $\mathbf{0}$, with $\beta(t)$ controlling the rate.

The diffusion term $\sqrt{\beta(t)}\,d\mathbf{W}_t$ adds fresh Gaussian kicks at the *same* rate. The two forces are calibrated (specifically, with the $\frac{1}{2}$ in the drift and the $\sqrt{\cdot}$ on the noise) to balance into a **mean-reverting process** whose stationary distribution is exactly $\mathcal{N}(\mathbf{0}, I)$. This is the Ornstein–Uhlenbeck process with time-varying rate $\beta(t)$. So no matter where $\mathbf{z}_0$ starts, after running for long enough, $\mathbf{z}_t \to \mathcal{N}(\mathbf{0}, I)$ — which is exactly what we wanted of the forward process.

The reverse-time SDE

A general fact due to Anderson (1982) is that **every diffusion SDE has a reverse-time SDE** that runs from noise back to data, and that this reverse SDE is determined entirely by the **score function** $\nabla_{\mathbf{z}_t} \log q(\mathbf{z}_t)$ of the time-marginal distribution $q(\mathbf{z}_t) \equiv \int q(\mathbf{z}_t \mid \mathbf{x}) q_{\text{data}}(\mathbf{x}) d\mathbf{x}$ at noise level t. Concretely:

$$d\mathbf{z}_t \;=\; \underbrace{\left[-\tfrac{1}{2}\beta(t)\,\mathbf{z}_t \;-\; \beta(t)\,\nabla_{\mathbf{z}_t}\log q(\mathbf{z}_t)\right]dt}_{\text{drift: original drift + score correction}} \;+\; \underbrace{\sqrt{\beta(t)}\,d\bar{\mathbf{W}}_t}_{\text{reverse-time diffusion}}$$

Two new things here:

- The **drift now has a score correction term**: $-\beta(t)\,\nabla \log q(\mathbf{z}_t)$. This is exactly the gradient field that points "toward the data" at noise level t. It's what makes the dynamics walk back up the density rather than diffusing further.
- The **bar over $\bar{\mathbf{W}}_t$** denotes a **reverse-time Brownian motion**: a Wiener process whose increments are iid Gaussian as time runs *backwards* (from T down to 0).

The takeaway is that the continuous-time view *reveals* what we need to learn: **the score function at every noise level**.

Discretizing the reverse-time SDE

To actually *sample* from the reverse SDE, we have to discretize it. The standard recipe is **Euler-Maruyama**: replace dt with Δt and $d\bar{\mathbf{W}}_t$ with $\sqrt{\Delta t}\,\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$. Stepping backward in time from t to $t - \Delta t$ (so $dt \to -\Delta t$, and the drift terms flip sign):

$$\mathbf{z}_{t-\Delta t} \;=\; \mathbf{z}_t \;+\; \left[\tfrac{1}{2}\,\beta(t)\,\mathbf{z}_t \;+\; \beta(t)\,\nabla_{\mathbf{z}_t}\log q(\mathbf{z}_t)\right]\,\Delta t \;+\; \sqrt{\beta(t)\,\Delta t}\,\boldsymbol{\epsilon}$$

Equivalently, this says $\mathbf{z}_{t-\Delta t}$ is sampled from a Gaussian whose mean is a linear combination of $\mathbf{z}_t$ and the **score** at this noise level:

$$\mathbf{z}_{t-\Delta t} \;\sim\; \mathcal{N}\big(\,a(t)\,\mathbf{z}_t \;+\; b(t)\,\nabla_{\mathbf{z}_t}\log q_t(\mathbf{z}_t),\;\; \sigma^2(t)\,I\big)$$

where $a(t), b(t), \sigma^2(t)$ are some functions of t determined by the noise schedule.

Look at the structure: a Gaussian reverse-step, mean linear in $\mathbf{z}_t$ and the score, variance set by the schedule. This is the **same form as the discrete reverse step we set up at the beginning of the lecture**, $p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}_t, t),\, \sigma_q^2(t)\,I)$ except now the mean is *written in terms of the score* rather than in terms of $\mathbf{x}$ (or $\boldsymbol{\epsilon}$).

The score function perspective

The **score function** of a distribution $q(\mathbf{x})$ is

$$s(\mathbf{x}) \;\equiv\; \nabla_{\mathbf{x}}\log q(\mathbf{x})$$

It is a vector field: at every $\mathbf{x}$, it points in the direction of steepest increase of $\log q$.

Two important properties.

- It **doesn't involve the normalization constant** of q. Writing $q(\mathbf{x}) = \tilde{q}(\mathbf{x})/Z$ with intractable Z, we can see that the gradient kills the $\log Z$ term. For high-dimensional $\mathbf{x}$ this is nice.
- It tells you exactly how to **sample** from q, via **Langevin dynamics**:

$$\mathbf{x}_{t+1} \;=\; \mathbf{x}_t \;+\; c\,\nabla\log q(\mathbf{x}_t) \;+\; \sqrt{2c}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, I)$$

which defines a Markov chain stationary distribution is q. So if you know the score function, you could sample. And you can *learn* the score by minimizing the Fisher divergence $\mathbb{E}_q[\|s_\theta(\mathbf{x}) - \nabla\log q(\mathbf{x})\|^2]$. Even though you don't have access to the true score, **score matching** techniques let you minimize this objective using just samples.

Tweedie's formula: noise prediction *is* score estimation

In the last sections we found that we could equivalently predict the mean μ_θ, predict the data $\hat{\mathbf{x}}_\theta$ or predict the noise $\hat{\epsilon}_\theta$. The SDE perspective suggests we could also predict the score $\hat{s}_\theta$.

Indeed a classic result known as **Tweedie's formula** (Robbins, 1956; Efron, 2011) explains why. For a Gaussian variable $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ with unknown mean $\boldsymbol{\mu}$,

$$\mathbb{E}_{\boldsymbol{\mu}\sim q(\boldsymbol{\mu}\mid\mathbf{z})}\big[\boldsymbol{\mu}\big] \;=\; \mathbf{z} \;+\; \Sigma\,\nabla_{\mathbf{z}}\log q(\mathbf{z})$$

Here $q(\mathbf{z}) = \int q(\mathbf{z} \mid \mu) g(\mu) d\mu$ is the marginal of $\mathbf{z}$ after integrating out $\boldsymbol{\mu}$ against its prior g (this could be *any* prior).

If we apply this to our setting, where $\mathbf{z}_t \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x},\; (1-\bar{\alpha}_t)\,I)$ with $\mathbf{x} \sim q_{\text{data}}$, Tweedie gives us

$$\mathbb{E}_{\mathbf{x}\sim q(\mathbf{x}\mid\mathbf{z}_t)}\big[\sqrt{\bar{\alpha}_t}\,\mathbf{x}\big] \;=\; \mathbf{z}_t \;+\; (1-\bar{\alpha}_t)\,\nabla_{\mathbf{z}_t}\log q(\mathbf{z}_t) \qquad (*)$$

Now use the reparameterization $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$, which rearranges to the **deterministic identity**

$$\sqrt{\bar{\alpha}_t}\,\mathbf{x} \;=\; \mathbf{z}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$$

Take the conditional expectation of *both sides* given $\mathbf{z}_t$. The LHS is exactly the LHS of $(*)$. The RHS becomes $\mathbf{z}_t - \sqrt{1-\bar{\alpha}_t}\,\mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{z}_t]$ (since $\mathbf{z}_t$ is constant given $\mathbf{z}_t$):

$$\mathbb{E}_{\mathbf{x}\sim q(\mathbf{x}\mid\mathbf{z}_t)}\big[\sqrt{\bar{\alpha}_t}\,\mathbf{x}\big] \;=\; \mathbf{z}_t \;-\; \sqrt{1-\bar{\alpha}_t}\,\mathbb{E}_{\boldsymbol{\epsilon}\sim q(\boldsymbol{\epsilon}\mid\mathbf{z}_t)}\big[\boldsymbol{\epsilon}\big] \qquad (**)$$

Equate $(*)$ and $(**)$ and solve for the score:

$$\boxed{\nabla_{\mathbf{z}_t} \log q(\mathbf{z}_t) \;=\; -\frac{\mathbb{E}\big[\boldsymbol{\epsilon} \mid \mathbf{z}_t\big]}{\sqrt{1-\bar{\alpha}_t}}}$$

So when we train $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{z}_t, t)$ to minimize $\mathbb{E}\|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{z}_t, t)\|^2$ over the joint distribution of $(\mathbf{x}, \boldsymbol{\epsilon}, \mathbf{z}_t)$, the *Bayes-optimal* predictor is precisely the conditional mean:

$$\hat{\boldsymbol{\epsilon}}^*_\theta(\mathbf{z}_t, t) \;=\; \mathbb{E}\big[\boldsymbol{\epsilon} \mid \mathbf{z}_t\big]$$

Substituting back into the boxed equation:

$$\nabla_{\mathbf{z}_t} \log q(\mathbf{z}_t) \;=\; -\frac{\hat{\boldsymbol{\epsilon}}^*_\theta(\mathbf{z}_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

And so we could have instead simply parameterized a neural network $s_\theta(\mathbf{z}_t, t)$ to predict the score!

Three problems with plain score matching, and the diffusion fix

If we could just learn the score of $q(\mathbf{x})$ directly, we'd be done. But there are three problems:

1. **Manifold problem.** Real data (e.g. images) lies on a low-dimensional manifold of $\mathbb{R}^D$. Off the manifold, $q = 0$ and the score is undefined.
2. **Low-density problem.** Score matching is an expectation under q, so we get no training signal in low-density regions. But Langevin sampling has to *start* in those regions (from random initializations).
3. **Mode-mixing problem.** Langevin dynamics may take too long to mix between separated modes.

All three problems are solved by adding multiple levels of Gaussian noise to the data. At high levels of noise, the distribution covers all of $\mathbb{R}^D$ and bridges separated modes. So we learn the score *at every noise level*: $s_\theta(\mathbf{z}_t, t) \approx \nabla \log q(\mathbf{z}_t)$, where q is the data distribution corrupted to noise level t. To generate, we then run Langevin starting from the noisiest level and *anneal* t downward.